University Institute of Computing

UNIVERSITY INSTITUTE OF COMPUTING

PYTHON PROGRAMMING

CASE STUDY REPORT

Sales Data Analysis Using Python

Submitted by	Craig Mapungwana
UID	24BCA10325
Section	24BCA(3B)
Department	University Institute of Computing
University	Chandigarh University

Academic Case Study

1. Introduction

Retail businesses generate sales records every day. These records contain useful information about products sold, quantities, prices and the money earned. When a store keeps this information only in handwritten registers or scattered spreadsheets, it can be difficult to identify which products perform best or to calculate revenue accurately.

This case study demonstrates how Python can be used to analyse a small retail store's sales data. The program stores a sample sales dataset, calculates revenue for each product, finds the best-selling product by units sold, identifies the product generating the highest revenue, and displays the results in tables and charts. The example uses sample data created for learning purposes; it does not represent the records of an actual store.

1.1 Problem Statement

A retail store wants to analyse its sales data to identify the best-selling products and calculate total revenue. The store needs a simple and reliable program that can process product names, quantities sold and selling prices, then present meaningful results that support business decisions.

1.2 Objectives

• To create and organise a small retail sales dataset using Python.

• To calculate revenue generated by each product.

• To calculate total units sold and total store revenue.

• To identify the best-selling product based on quantity sold.

• To identify the product that generated the highest revenue.

• To present the findings in a clear tabular and graphical form.

2. Tools and Technologies

Tool	Purpose
Python 3	Main programming language used to process and analyse the data.
Pandas	Organises sales records in a DataFrame and performs calculations.
Matplotlib	Creates charts to compare product sales and revenue.
Jupyter Notebook / VS Code	Environment in which the program can be written and executed.

The program can also be run without Pandas or Matplotlib by using basic Python lists and loops. In this report, Pandas is used for data handling and Matplotlib is used for visualisation.

3. Dataset Description

A sample dataset of six products is used. Each record contains the product name, quantity sold and selling price per unit. Revenue for an individual product is calculated by multiplying quantity sold by selling price per unit.

Product	Quantity sold	Price per unit (■)	Revenue (■)
Wireless Mouse	45	20	900

Product	Quantity sold	Price per unit (■)	Revenue (■)
USB Keyboard	30	25	750
Laptop Stand	20	35	700
USB-C Hub	15	50	750
Headphones	12	80	960
Webcam	10	60	600
Total	**132**	—	**4,660**

The dataset shows 132 units sold and total revenue of ■4,660. Headphones generate the highest revenue (■960), while the Wireless Mouse has the highest number of units sold (45). These are two different measures of product performance.

4. Methodology and Calculations

4.1 Data preparation

The product records are entered into a Python dictionary and converted into a Pandas DataFrame. A new column named Revenue is calculated by multiplying Quantity by Price. This makes it possible to compare the performance of products using the same calculation method.

4.2 Key calculations

• **Product revenue** = Quantity Sold × Price per Unit

• **Total revenue** = Sum of revenue for all products

• **Total units sold** = Sum of quantities sold for all products

• **Best-selling product** = Product with the maximum quantity sold

• **Highest-revenue product** = Product with the maximum revenue

For example, Wireless Mouse revenue is 45 × ■20 = ■900. Headphones revenue is 12 × ■80 = ■960. The program uses the maximum value in the Quantity column to find the best-selling product, and the maximum value in the Revenue column to find the highest-revenue product.

5. Python Program

The following program performs the analysis from start to finish. It prints the sales table, calculates summary values, identifies the two leading products and generates two bar charts.

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample retail sales data
data = {
    "Product": [
        "Wireless Mouse", "USB Keyboard", "Laptop Stand",
        "USB-C Hub", "Headphones", "Webcam"
    ],
    "Quantity": [45, 30, 20, 15, 12, 10],
    "Price": [20, 25, 35, 50, 80, 60]
}

# Create a DataFrame
df = pd.DataFrame(data)
```

```
# Calculate revenue for every product
df["Revenue"] = df["Quantity"] * df["Price"]

# Display the complete sales table
print("SALES DATA")
print(df.to_string(index=False))

# Calculate overall sales figures
total_units = df["Quantity"].sum()
total_revenue = df["Revenue"].sum()

# Find the best-selling product by quantity
best_selling = df.loc[df["Quantity"].idxmax()]

# Find the product with the highest revenue
top_revenue = df.loc[df["Revenue"].idxmax()]

print("\\nSALES SUMMARY")
print("Total units sold:", total_units)
print("Total revenue: Rs.", total_revenue)
print("Best-selling product:", best_selling["Product"])
print("Units sold:", best_selling["Quantity"])
print("Highest-revenue product:", top_revenue["Product"])
print("Revenue: Rs.", top_revenue["Revenue"])

# Chart 1: revenue by product
plt.figure(figsize=(9, 5))
plt.bar(df["Product"], df["Revenue"])
plt.title("Revenue by Product")
plt.xlabel("Product")
plt.ylabel("Revenue (Rs.)")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()

# Chart 2: quantity sold by product
plt.figure(figsize=(9, 5))
plt.bar(df["Product"], df["Quantity"])
plt.title("Units Sold by Product")
plt.xlabel("Product")
plt.ylabel("Units Sold")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()
```

6. Results and Output

When the program is executed, it displays the calculated sales table and a summary of the store's performance. The main results from the sample dataset are shown below.

Measure	Result
Total units sold	132 units
Total revenue	■4,660
Best-selling product by quantity	Wireless Mouse — 45 units
Highest-revenue product	Headphones — ■960

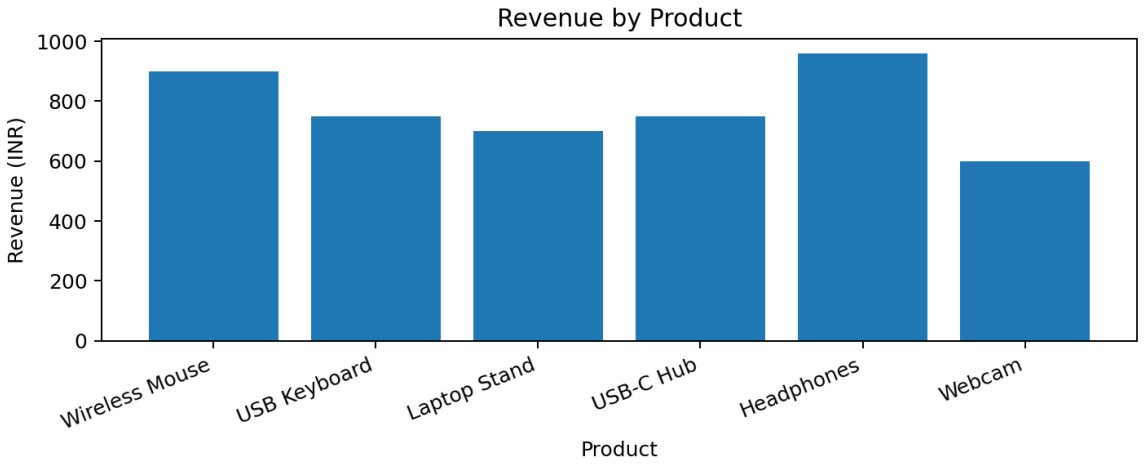


Figure 1: Revenue generated by each product.

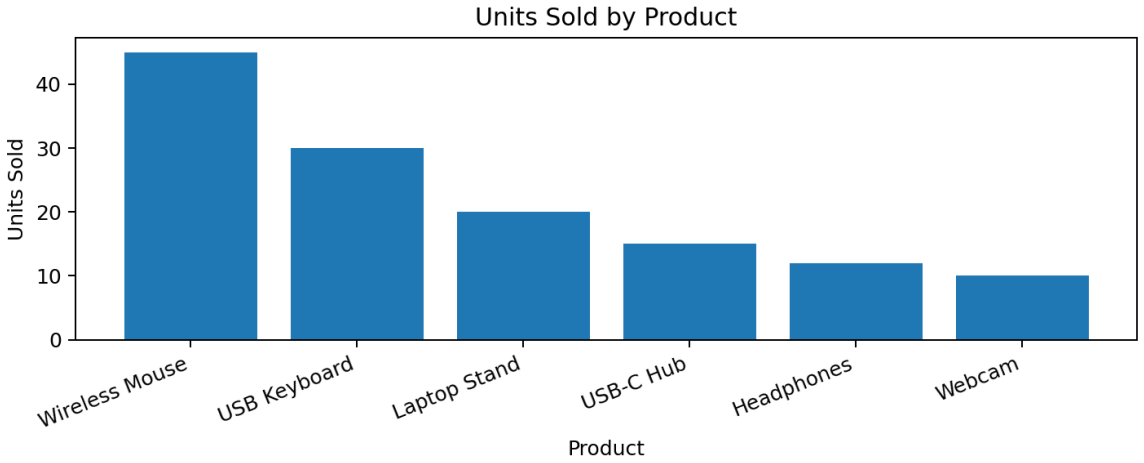


Figure 2: Number of units sold for each product.

7. Interpretation and Discussion

The analysis indicates that the product with the highest sales volume is not necessarily the product that generates the most revenue. Wireless Mouse leads in quantity, with 45 units sold, but its lower unit price results in ■900 revenue. Headphones sell only 12 units but generate ■960 because their selling price is higher.

The store can use these findings when planning stock purchases and promotions. It may need to monitor Wireless Mouse inventory closely because it sells in the greatest quantity. It can also review the pricing, promotion and stock availability of Headphones because they contribute the highest revenue in this sample.

The USB Keyboard and USB-C Hub each generate ■750. The Webcam has the lowest quantity sold (10 units) and the lowest revenue (■600). This does not automatically mean that the Webcam should be discontinued; the store should also consider profit margin, customer demand, purchase cost and seasonal trends before making that decision.

8. Advantages and Limitations

8.1 Advantages

• Calculations are performed consistently and reduce manual arithmetic errors.

• The program quickly summarises multiple product records.

• Charts make it easier to compare sales volume and revenue.

• The dataset can be expanded by adding more products or sales records.

• The results can support stock planning and business decisions.

8.2 Limitations

• The dataset is small and illustrative, not real transaction data.

• The program analyses revenue but does not calculate profit because product costs and operating expenses are not provided.

• It does not include dates, so monthly or seasonal sales trends cannot be studied.

• The current version uses data entered in the code rather than importing a live point-of-sale database.

• It does not automatically detect invalid or missing records.

9. Future Scope

The project can be improved by reading sales records from CSV or Excel files, adding transaction dates, and producing daily, weekly or monthly reports. A future version could calculate profit by including the purchase cost of each product, show low-stock alerts, and provide a simple graphical user interface. It could also validate input values to prevent negative quantities or invalid prices.

10. Conclusion

This case study demonstrates how Python can help a retail store analyse sales information in a structured and efficient way. Using a sample dataset of six products, the program calculates revenue for each product, total units sold and total revenue. It also identifies the best-selling product by quantity and the product generating the highest revenue.

The results show total sales of 132 units and total revenue of ■4,660. Wireless Mouse is the best-selling product by quantity, while Headphones generate the highest revenue. The charts make these findings easier to understand. Overall, the project shows how basic Python programming, data analysis and visualisation can turn raw sales records into useful business information.

References

• Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

• Pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/

• Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/